
CORRECTION DE DISTORSION D'IMAGE PAR RECHERCHE ITÉRATIVE DU COEFFICIENT DE DISTORSION

Ce projet a été développé dans le cadre du TIPE de la session 2026 pour les concours d'entrée aux Grandes Écoles. Il est basé sur les travaux [1] et [2] de DANIEL SANTANA-CEDRÉS, LUIS GOMEZ, MIGUEL ALEMÁN-FLORES, AGUSTIN SALGADO, JULIO ESCLARIN, LUIS MAZORRA et LUIS ALVAREZ.

Présentation

La distorsion est une aberration géométrique (ou monochromatique) ne dépendant que de la taille de l'objet pour un système optique donné. Les distorsions d'images sont présentes dans toutes les caméras en raison de leur construction. Aujourd'hui sur les smartphones, ce phénomène est de plus en plus présent avec la démocratisation des objectifs à ultra-grand angle d'entrée de gamme. Pour exemple dans ce TIPE, nous allons utiliser une image et simuler sa distorsion avec le paramètre $p = 0,6$.

Modèle de distorsion

Le modèle de distorsion utilisé dans ce TIPE sera celui du modèle de division. Pour un pixel de l'image représenté par les coordonnées $(\hat{x}, \hat{y})$ de l'image sans distorsion, on a les coordonnées (x, y) du pixel sur lequel on a appliqué la distorsion,

$$\begin{aligned}x &= x_c + (\hat{x} - x_c)(1 + k_1 r^2) \\ y &= y_c + (\hat{y} - y_c)(1 + k_1 r^2)\end{aligned}$$

avec $r = \sqrt{(x - x_c)^2 + (y - y_c)^2}$, (x_c, y_c) les coordonnées du centre de distorsion et k_1 le coefficient de distorsion. On supposera que le centre de distorsion est également le centre de l'image. C'est une bonne approximation dans la plupart des cas. On peut alors appliquer une correction à chacun des pixels d'une image corrigée,

$$\begin{aligned}\hat{x} &= x_c + \left(\frac{1}{1 + k_1 r^2}\right)(x - x_c) \\ \hat{y} &= y_c + \left(\frac{1}{1 + k_1 r^2}\right)(y - y_c)\end{aligned}$$

De plus, on utilisera une normalisation p du paramètre k_1 pour qu'il soit proportionnel à la taille de l'image,

$$\begin{aligned}k_1 &= -\frac{p}{(1 + p)r_{\max}^2} \\ p &= -\frac{k_1 r_{\max}^2}{1 + k_1 r_{\max}^2}\end{aligned}$$

avec $r_{\max} = \sqrt{d_x^2 + d_y^2}$ et d_x et d_y respectivement les dimensions en largeur et en hauteur de l'image. Pour assurer la bijection de l'expression de p , on doit avoir que $p \geq -0,5$.

Pour corriger l'angle θ du gradient de l'image après distorsion, on note $x_{\text{ori}} = x + \cos(\theta)$, $y_{\text{ori}} = y + \sin(\theta)$, ainsi que $\hat{x}_{\text{ori}}$, $\hat{y}_{\text{ori}}$, leurs valeurs après l'application du modèle pour inverser la distorsion. Puis, $a = \hat{x}_{\text{ori}} - \hat{x}$, $b = \hat{y}_{\text{ori}} - \hat{y}$ et donc l'orientation corrigée après distorsion $\hat{\theta}$ de (x, y)

$$\hat{\theta} = \arccos\left(\frac{a}{a^2 + b^2}\right)$$

Principe de détection du paramètre p

La distorsion est marquée par la courbure de lignes qui étaient droites sur l'image sans distorsion. L'algorithme prend donc avantage de cette caractéristique pour corriger l'image. La présence de lignes droites dans le paysage est donc nécessaire pour avoir de bons résultats. Pour pouvoir détecter le bon paramètre de distorsion, on va rechercher itérativement le bon paramètre p en appliquant une correction de distorsion pour chaque pixel, qui voteront ensuite chacun pour une équation de ligne suivant une transformée de Hough. Le score de ce paramètre sera la somme des scores des lignes détectées sur l'image et il suffira de prendre le paramètre qui conduit au score le plus élevé pour la transformée de Hough.

Algorithme

L'algorithme va dans un premier temps, détecter les contours de l'image en utilisant la méthode de Canny, puis dans un second temps, utiliser une transformée de Hough améliorée pour détecter les lignes droites.

1. Flou Gaussien

Un flou Gaussien est appliqué à l'image pour pouvoir éliminer d'éventuels problèmes liés au bruit de l'image. Pour cela, on applique un noyau m par produit de convolution à l'image,

$$m = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

avec 4 représentant le poids du pixel auquel on applique le noyau. Ainsi la valeur de la couleur d'un pixel est égale à la somme des valeurs de chaque pixel multipliées par la valeur du noyau correspondant puis divisée par la somme du noyau (soit ici 16). L'image est stockée dans un tableau de taille t , pour un pixel (x, y) on associe son index i :

$$t = channel \times d_x \times d_y \\ i = (y \times d_x + x) \times channel + c$$

avec $channel = 3$ pour une image RGB et c correspondant à la couleur voulue (0 pour le rouge, 1 pour le vert et 2 pour le bleu). On utilisera par la suite le même système d'indice pour stocker des informations dans des tableaux en deux dimensions, en ajustant bien évidemment $channel$ et la taille du tableau en conséquence. Seuls les pixels au bord de l'image sont recopiés sans modification, car le noyau déborderait en dehors de l'image.

2. Gradient de l'image

De la même façon, deux noyaux sont appliqués, un pour le gradient dans le sens des x avec m_x et l'autre dans le sens des y avec m_y ,

$$m_x = 0.5 \begin{bmatrix} -\frac{2-\sqrt{2}}{2} & 0 & \frac{2-\sqrt{2}}{2} \\ -(\sqrt{2}-1) & 0 & \sqrt{2}-1 \\ -\frac{2-\sqrt{2}}{2} & 0 & \frac{2-\sqrt{2}}{2} \end{bmatrix} \\ m_y = 0.5 \begin{bmatrix} -\frac{2-\sqrt{2}}{2} & -(\sqrt{2}-1) & -\frac{2-\sqrt{2}}{2} \\ 0 & 0 & 0 \\ \frac{2-\sqrt{2}}{2} & \sqrt{2}-1 & \frac{2-\sqrt{2}}{2} \end{bmatrix}$$

On utilise le même système de coordonnées d'index que précédemment avec $channel = 2$, car on stocke le gradient en x , (g_x) , en $c = 0$ et le gradient en y , (g_y) , en $c = 1$. De même, seuls les pixels du bord ne sont pas modifiés. La norme du gradient g de l'image est donnée en (x, y) par $|g(x, y)|$ et l'angle par θ :

$$|g(x, y)| = \sqrt{g_x^2(x, y) + g_y^2(x, y)} \\ \theta = \text{atan2}(g_y(x, y), g_x(x, y))$$

si l'angle est négatif, on lui rajoute π pour que $\theta \in [0; \pi]$. Attention, l'angle donné est celui du gradient et est décalé de $\frac{\pi}{2}$ par rapport à la direction de la ligne sur laquelle se trouve le pixel (x, y) .

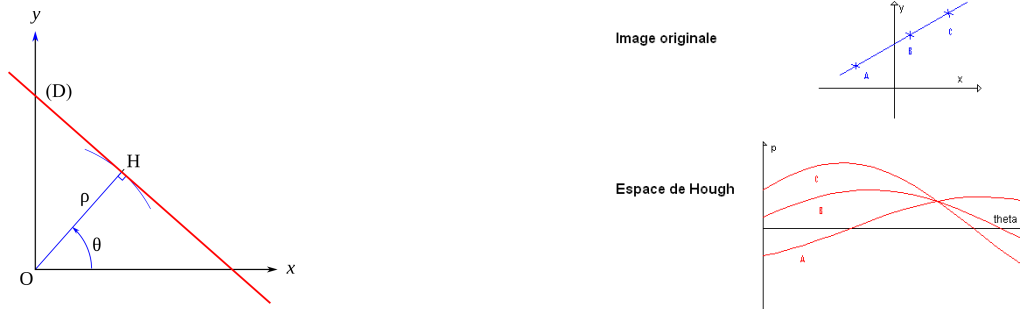
3. Construction des contours de l'image

Pour construire les contours de l'image, on va d'abord rechercher la norme maximale du gradient, puis introduire un seuil haut et un seuil bas pour les contours de l'image. Expérimentalement, on fixe le seuil haut à 40% de la norme maximale et le seuil bas à 1%. On explore ensuite tous les pixels. Un pixel dépassant le seuil haut est automatiquement considéré comme un point d'un contour, puis on explore récursivement les voisins du pixel. Si un pixel dépassant le seuil bas est adjacent à un pixel déjà classé comme un point d'un contour, alors lui aussi est considéré comme un point d'un contour. Les seuils ici sont choisis plutôt sélectifs pour réduire le temps de calcul lors de la transformée de Hough.

4. Transformée de Hough améliorée

La transformée de Hough est un algorithme permettant de détecter les lignes droites dans une image avec sa version standard. Ici, les lignes droites sont courbées sous l'effet de la distorsion. On doit donc appliquer une distorsion inverse sur les pixels afin qu'ils votent pour la ligne leur correspondant. On utilise pour cela l'équation de droite

$$x \cos(\theta) + y \sin(\theta) + \rho = 0$$



Images modifiées de Wikipédia https://fr.wikipedia.org/wiki/Transform%C3%A9e_de_Hough

avec θ l'angle de la droite perpendiculaire à la ligne votée par rapport à l'axe x et ρ la distance entre l'intersection de la droite à la ligne et l'origine. On construit alors un tableau de vote, initialisé à 0, de taille $2r_{\max}\theta_{\dim}$. On doit le multiplier par 2, car $\rho \in [-r_{\max}; r_{\max}]$ et $\theta \in [0; \pi]$, on découpe alors π en $\theta_{\dim}$ nombre de parts que l'on fera correspondre à θ en degré. En pratique, on utilisera $\theta_{\dim} = 4000$ fixé.

Dans la transformée de Hough standard, on connaît la position (x, y) du pixel et on vote pour chaque paramètre ρ et θ satisfaisant l'équation de la droite. Cela correspond à voter pour toutes les droites qui passent par le point (x, y) . Ici, grâce au gradient, il est possible de connaître ρ et θ et on évite ainsi de voter pour tous les paramètres ρ et θ satisfaisants l'équation. Soit d_l la distance du point par rapport à la ligne, chaque pixel va ajouter à la ligne pour laquelle il vote le score de

$$\frac{1}{1 + d_l}$$

À la fin du vote de tous les pixels, les cases de coordonnées ρ et θ les plus élevées seront alors considérées comme des lignes avec leurs paramètres respectifs (ρ, θ) . Lors de l'ajout d'une ligne à l'ensemble des résultats (lignes détectées), on s'assure qu'elle est différente des autres avec une distance supérieure à 2 pixels des autres et un angle différent d'au moins 2° .

Si p ne correspond pas à la distorsion de l'image, on ne détectera que des bouts de lignes, donc un score faible. À l'inverse, si p est correct, on détectera tous les points des lignes, soit un score plus important. Le paramètre p devrait alors correspondre à l'exécution de la transformée de Hough avec le plus grand score. Le nombre de lignes à détecter reste constant lors des exécutions pour que l'on puisse comparer les scores. Les lignes tracées sur l'image seront en blanc, parmi les pixels, ceux étant des contours seront marqués en rouge.

Pseudo-code de la transformée de Hough améliorée

```

1: for all  $p \in [p_{\min}; p_{\max}]$  do
2:   for all  $(x, y) \in \text{image}$  do
3:      $\alpha = \text{corrige}(\theta, p)$ 
4:      $(\hat{x}, \hat{y}) = \text{corrige}((x, y), p)$ 
5:     for all  $\beta \in [\alpha - \alpha_{\text{res}}; \alpha + \alpha_{\text{res}}]$  do
6:        $\rho = \lfloor -(\hat{x} \cos(\beta) + \hat{y} \sin(\beta)) \rfloor$ 
7:       for all  $\text{dist} \in [\rho - \text{dist}_{\text{res}}; \rho + \text{dist}_{\text{res}}]$  do
8:          $d_l = |\hat{x} \cos(\beta) + \hat{y} \sin(\beta) + \text{dist}|$  (distance à la ligne)
          score =  $\frac{1}{1 + d_l}$ 
          vote( $(\hat{x}, \hat{y}), \text{score}$ )
9:       end for
10:    end for
11:  end for
12: end for

```

avec $\text{dist}_{\text{res}} = 2$, $\alpha_{\text{res}} = 2^\circ$

5. Correction de l'image

Après détection itérative du paramètre, on obtient p , qui sera ensuite utilisé pour remplacer chaque pixel de l'image en utilisant les équations définies plus tôt. En regardant la position du pixel $(0,0)$ après distorsion pour des paramètres $p > 0$, on rognera l'image corrigée pour éviter l'apparition des zones noires sur les bords de l'image.

Modifications du modèle proposé

Pour pouvoir détecter automatiquement une ligne, on définit un seuil bas pour le score nécessaire pour pouvoir considérer une ligne comme significative. Expérimentalement, on trouve que le seuil s_y pour les lignes orientées à la verticale $\left(\theta \in \left[0, \frac{\pi}{4}\right] \cup \left[\frac{3\pi}{4}, \pi\right]\right)$ et le seuil s_x pour les lignes orientées à l'horizontale $\left(\theta \in \left[\frac{\pi}{4}, \frac{3\pi}{4}\right]\right)$:

$$s_x = 0,04 \times d_x$$

$$s_y = 0,02 \times d_y$$

Les lignes ainsi détectées pour la première itération fixe le nombre de lignes à détecter pour le reste des boucles afin de garder la cohérence lors de la comparaison des scores entre chaque passage de boucle.

On fixe un minimum de 3 lignes ainsi qu'un maximum de 25 lignes, pour ne pas avoir des lignes trop petites. Si le nombre de lignes automatiquement détectées est strictement inférieur à 3, on prendra les 3 meilleures.

Après quelques tests, on remarque que le centre de l'image est peu affecté par la distorsion. En conséquence, les lignes du milieu font pencher le score total vers un paramètre p proche de 0. Pour minimiser l'impact de ces lignes, on divise le score de chaque pixel (x,y) se trouvant dans les zones suivantes par 10 :

$$\frac{4}{11}d_x \leq x \leq \frac{6}{11}d_x \quad \text{ou} \quad \frac{4}{11}d_y \leq y \leq \frac{6}{11}d_y$$

Les lignes qui passent de façon très proche du centre, subissent très peu voir pas du tout l'effet de distorsion. Ce ne sera généralement pas le cas pour des images réelles, les lignes seront parallèles pour des images de paysages urbains et des lignes sortiront des zones divisant le score. Plus généralement, les lignes passant par le milieu ne subissent aucune distorsion. Nous ne chercherons pas à minimiser ces cas, car on considère que dans les paysages urbains, il y a de nombreuses lignes qui ne passent pas par le centre. De plus, si très peu de lignes droites sont présentes, la distorsion est moins visible à l'œil nu.

Enfin, en même temps de corriger les pixels de l'image, on marque sur les emplacements de l'image de sortie, quels pixels ont été remplis. On exécute ensuite une autre boucle pour copier les données de pixels adjacents remplis à un pixel qui serait resté vide. Cela permet ainsi d'améliorer visuellement le résultat en introduisant cependant des artefacts dans l'image. Sinon, un "filet" apparaît sur l'image corrigée.

Exemples d'images

Exemple 1 : image synthétique

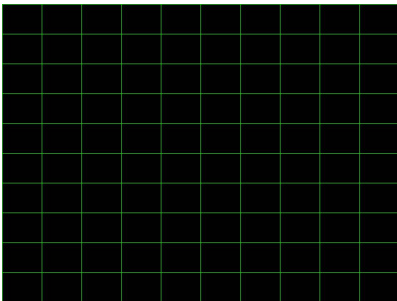


Image originale

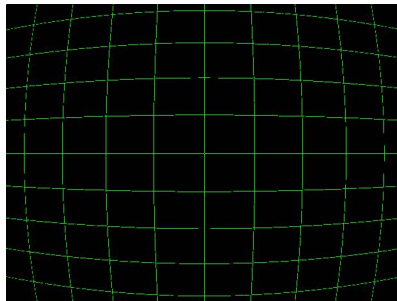


Image avec une distorsion $p = 0,6$

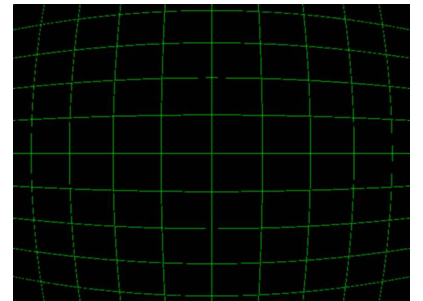
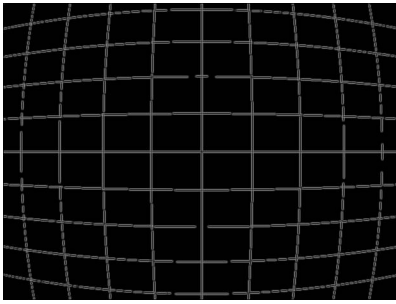
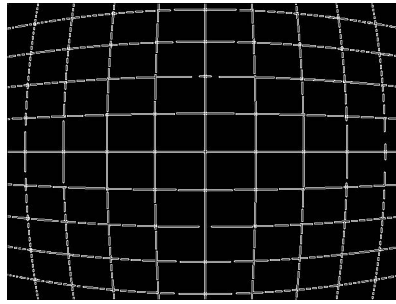


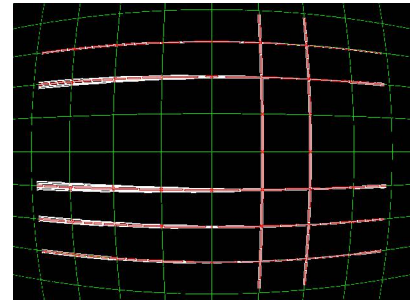
Image avec un flou gaussien appliqué



Gradient de l'image



Contours de l'image



Lignes détectées sur l'image

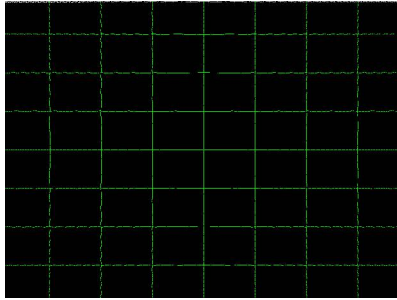


Image corrigée

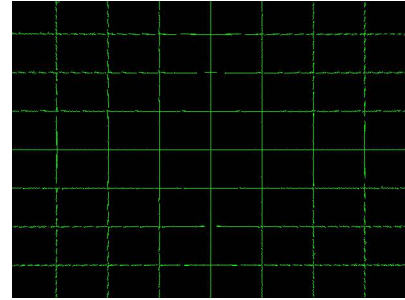


Image corrigée avec amélioration

Exemple 2 : image réelle modifiée



Image originale



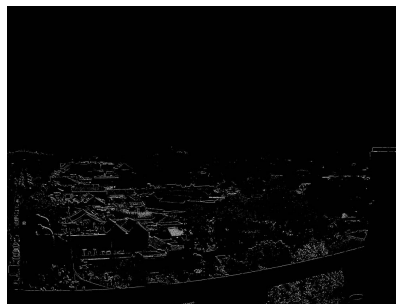
Image avec une distorsion $p = 0,6$



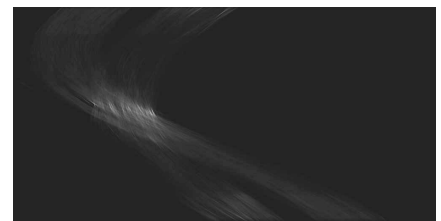
Image avec un flou gaussien appliqué



Gradient de l'image



Contours de l'image



Espace de vote des pixels (modifié)



Lignes détectées sur l'image avec distorsion appliquée



Image corrigée



Image corrigée avec amélioration

Exemple 3 : Pas de distorsion pour les lignes qui passent par le centre

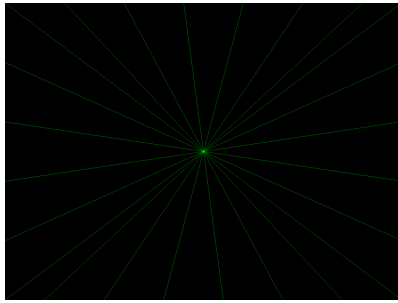


Image originale

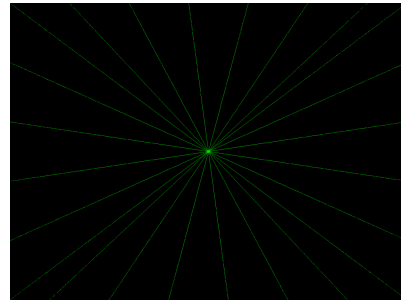


Image modifiée ($p = 2$)

Exemple 4 : atténuation des pixels du milieu



*Lignes détectées sans atténuation
des votes au milieu (Résultat :
 $p = 0$)*



*Lignes détectées avec atténuation
des votes au milieu (Résultat :
 $p = 0,35$)*



*Image corrigée avec atténuation
des votes au milieu*

Exemple 5 : image de paysage

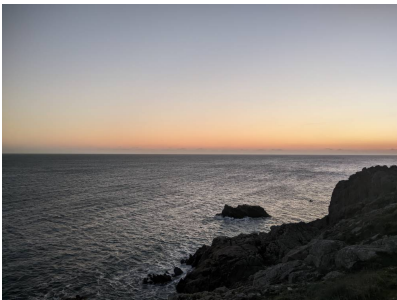


Image originale

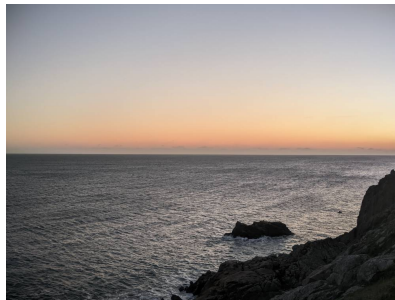
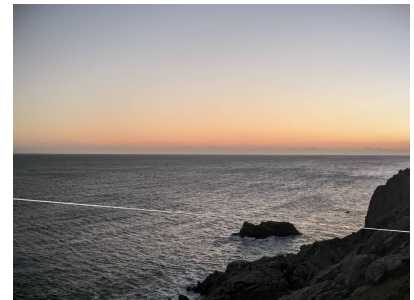


Image avec une distorsion $p = 1$



*Lignes détectées sur l'image
(Résultat : $p = 0$)*

Sortie standard du programme pour l'exemple 2

```
$ ./distort_image
Parametre p : 0.6
Loading the image Image/image.png
k1 : -0.00000005750743456
Writing the image as Image/distorted_image.png

$ ./programme
Paramètre p minimum : 0
Paramètre p maximum : 1
Arrêt automatique du programme : 1
Chargement de l'image Image/distorted_image.png...
Application du flou Gaussien...
Flou Gaussien appliqué
Temps pris pour le flou Gaussien : 0.377846 s
Écriture de l'image : Image/gaussian.png...
Calcul du gradient...
Gradient calculé
Temps pris pour le gradient : 0.341535 s
Écriture du gradient : Image/gradient.png...
```

```

Construction des contours...
Contours construits
Temps pris pour trouver les contours : 0.071978 s
Écriture des contours : Image/edge.png...
Transformée de Hough améliorée sur l'image...
Résultat pour p = 0.000000 score = 3601.958950 avec 25 lignes
Résultat pour p = 0.050000 score = 3886.704933 avec 25 lignes
Résultat pour p = 0.100000 score = 4133.887671 avec 25 lignes
Résultat pour p = 0.150000 score = 4452.680497 avec 25 lignes
Résultat pour p = 0.200000 score = 4518.986884 avec 25 lignes
Résultat pour p = 0.250000 score = 4665.029111 avec 25 lignes
Résultat pour p = 0.300000 score = 4572.196371 avec 25 lignes
Résultat pour p = 0.350000 score = 4420.610320 avec 25 lignes
Résultat pour p = 0.400000 score = 3968.905866 avec 25 lignes
Arrêt automatique du programme
Temps pris au total pour la transformée de Hough : 62.445871 s
Écriture de l'espace de vote : Image/voting_space.png...
Coefficient trouvé pour p = 0.250000
Écriture de l'image : Image/undistorted_image_without_correction.png...
Écriture de l'image : Image/undistorted_image.png...
Temps pris pour tracer les lignes sur l'image : 0.142476 s
Écriture de l'image : Image/lines_on_image.png...
Terminé avec p = 0.250000 !

```

Résultats

On choisit 6 images ci-dessous pouvant correspondre à des images réelles avec des lignes droites semblants être apparentent sur celles-ci.



Image 1

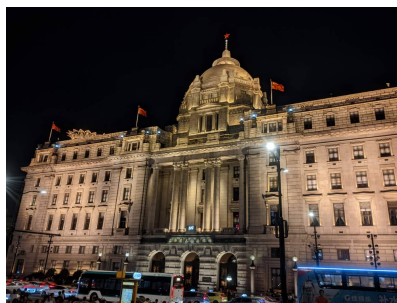


Image 2



Image 3



Image 4



Image 5



Image 6

On lance l'algorithme sur les distorsions $p = -0,3$; $p = -0,2$; $p = -0,1$; $p = 0,1$; $p = 0,4$; $p = 0,7$; $p = 0,9$ et $p = 2$.

Image / p	-0,3	-0,2	-0,1	0,1	0,4	0,7	0,9	2
Image 1	-0,3	-0,2	-0,1	0,1	0,2	0,25	0,35	0,4
Image 2	-0,45	-0,25	-0,15	0,1	0,25	0,3	0,35	0
Image 3	-0,2	0	-0,5	0,7	0,5	0,1	0,6	0
Image 4	-0,45	-0,5	-0,5	0,1	0,2	0,25	0,1	0,1
Image 5	0	-0,15	-0,1	0,05	0	0	0	0
Image 6	-0,3	-0,2	-0,1	0,05	0,2	0,25	0,35	0,4

Paramètre p détecté

Image / p	-0,3	-0,2	-0,1	0,1	0,4	0,7	0,9	2	Correction
Image 1	Oui	Oui	Oui	Oui	Oui	Oui	Oui	Oui	8/8
Image 2	Non	Oui	Oui	Oui	Oui	Oui	Oui	Non	6/8
Image 3	Oui	Non	Non	Non	Oui	Oui	Oui	Oui	5/8
Image 4	Non	Non	Non	Oui	Oui	Oui	Oui	Non	4/8
Image 5	Non	Oui	Oui	Oui	Non	Non	Non	Non	3/8
Image 6	Oui	Oui	Oui	Oui	Oui	Oui	Oui	Oui	8/8
Satisfaisant	3/6	4/6	4/6	5/6	5/6	5/6	5/6	3/6	

L'image semble avoir été corrigée

On remarque que même si l'image semble avoir des lignes droites sur l'image, la correction n'est pas toujours satisfaisante, en particulier pour les images 4 et 5 ce qui est assez surprenant. Comparé aux autres images, il semblerait que de grandes lignes droites sur l'image soient nécessaires pour donner des résultats satisfaisants. De plus, l'image 3 donne des résultats aléatoires, car cette image contient des éléments urbains ainsi qu'un arrière-plan naturel avec pas de grandes lignes visibles sur l'entièreté de l'image. Pour autant, les images corrigées ne semblent pas avoir de distorsion malgré un grand écart par rapport au paramètre d'entrée. Remarquons enfin que l'algorithme est le plus efficace dans l'intervalle $p = -0,2$ jusqu'à $p = 0,9$. En dessous et au-delà, l'algorithme détecte mal les lignes sur l'image.

Complexité

L'application du flou Gaussien est en

$$\mathcal{O}(d_x \times d_y)$$

Le calcul du gradient est en

$$\mathcal{O}(d_x \times d_y)$$

Le calcul des contours est en

$$\mathcal{O}(d_x \times d_y)$$

La transformée de Hough améliorée est en

$$\mathcal{O}\left(\left\lceil \frac{p}{p_{\text{step}}} \right\rceil \times \left\lceil \frac{\alpha_{\text{res}}}{\alpha_{\text{step}}} \right\rceil \times \left\lceil \frac{\text{dist}_{\text{res}}}{\text{dist}_{\text{step}}} \right\rceil \times d_x^2 \times d_y^2 \times \theta_{\text{dim}} \times \text{dist}_{\text{dim}}\right)$$

avec le tri des lignes en $\mathcal{O}(d_x \times d_y \times \theta_{\text{dim}} \times \text{dist}_{\text{dim}})$ (pour $\text{dist}_{\text{dim}} = 2 \left\lceil \sqrt{d_x^2 + d_y^2} \right\rceil$). En pratique, $p_{\text{step}} = 0,05$; $\alpha_{\text{step}} = 0,001$; $\text{dist}_{\text{step}} = 1$; $\alpha_{\text{res}} = 2^\circ$; $\text{dist}_{\text{res}} = 2$.

La correction de distorsion est en

$$\mathcal{O}(d_x \times d_y)$$

Tracer les lignes est en

$$\mathcal{O}(d_x \times d_y)$$

Utilisation du programme

Obtention d'un exécutable

Pour pouvoir exécuter le programme, il faudra télécharger le fichier [programme.c](#) ainsi que la [fonction de lecture](#) et la [fonction d'écriture](#). On placera les fichiers dans le même dossier et on compilera avec les commandes suivantes :

```
$ gcc -fsanitize=address -O3 -g programme.c -o programme -lm
```

ou

```
$ clang -fsanitize=address -O3 -g programme.c -o programme -lm
```

Attention, la présence du sanitizer est importante !

Exécution du programme

On créera là où se trouve l'exécutable programme, un dossier nommé **Image**. Dans celui-ci, on placera une image nommée **distorted_image.png** (le format de l'image doit être exactement en PNG).

Pour lancer le fichier sous Linux, on utilisera la commande

```
$ ./programme
```

Plusieurs paramètres vont vous être demandé.

Le paramètre **p** (flottant) doit être saisi dans l'ordre croissant et le minimum doit être supérieur ou égal à -0.5. Un paramètre **p** négatif correspond à une distorsion de type pincushion et un paramètre positif à une de type barrel. Il est recommandé d'avoir les paramètres **p** minimum et maximum de même signe ou nul pour de meilleurs résultats.

Le paramètre **Arrêt automatique du programme** (entier) correspond à la programmation de l'arrêt automatique du programme dans la détection du paramètre de distorsion. Il devra prendre la valeur 0 (désactivé) ou 1 (activé). Chaque paramètre se valide en utilisant la touche **entrée** du clavier.

Exemple de données d'entrée

```
Paramètre p minimum : 0
Paramètre p maximum : 1.1
Arrêt automatique du programme : 1
```

Résultats du programme

Le résultat de la détection du paramètre de distorsion sera indiqué en dernière ligne de la sortie sur le terminal. Les différentes étapes de l'algorithme y sont indiqués ainsi le score de la transformée de Hough pour chaque paramètre **p**.

Le programme aura comme effet de produire les images des différents stades de l'algorithme dans le dossier **Image** : **gaussian.png** (flou Gaussien), **gradient.png** (gradient de l'image), **edge.png** (contours de l'image), **voting_space.png** (espace de vote pour la transformée améliorée de Hough), **lines_on_image.png** (lignes détectées sur l'image pour le paramètre de distorsion final **p** détecté), **undistorted_image_without_correction.png** (l'image corrigée sans amélioration visuelle, si elle est calculée) et enfin, **undistorted_image.png** (l'image corrigée sans distorsion avec éventuellement, une amélioration visuelle).

Exemple d'exécution

```
$ ./programme
Paramètre p minimum : 0
Paramètre p maximum : 1.1
Arrêt automatique du programme : 1
Chargement de l'image Image/distorted_image.png...
Application du flou Gaussien...
Flou Gaussien appliqué
Temps pris pour le flou Gaussien : 0.340244 s
Écriture de l'image : Image/gaussian.png...
Calcul du gradient...
Gradient calculé
Temps pris pour le gradient : 0.329030 s
Écriture du gradient : Image/gradient.png...
Construction des contours...
Contours construits
Temps pris pour trouver les contours : 0.068301 s
Écriture des contours : Image/edge.png...
Transformée de Hough améliorée sur l'image...
Résultat pour p = 0.000000 score = 3601.958950 avec 25 lignes
Résultat pour p = 0.050000 score = 3886.704933 avec 25 lignes
Résultat pour p = 0.100000 score = 4133.887671 avec 25 lignes
Résultat pour p = 0.150000 score = 4452.680497 avec 25 lignes
Résultat pour p = 0.200000 score = 4518.986884 avec 25 lignes
Résultat pour p = 0.250000 score = 4665.029111 avec 25 lignes
Résultat pour p = 0.300000 score = 4572.196371 avec 25 lignes
Résultat pour p = 0.350000 score = 4420.610320 avec 25 lignes
Résultat pour p = 0.400000 score = 3968.905866 avec 25 lignes
Arrêt automatique du programme
Temps pris au total pour la transformée de Hough : 63.834160 s
```

```
Écriture de l'espace de vote : Image/voting_space.png...  
Coefficient trouvé pour  $p = 0.250000$   
Écriture de l'image : Image/undistorted_image_without_correction.png...  
Écriture de l'image : Image/undistorted_image.png...  
Temps pris pour tracer les lignes sur l'image : 0.141361 s  
Écriture de l'image : Image/lines_on_image.png...  
Terminé avec  $p = 0.250000$  !
```

MCOT

Correction de distorsion d'image par recherche itérative du coefficient de distorsion

Les distorsions d'images sont présentes dans toutes les caméras en raison de leur construction. Aujourd'hui avec les smartphones, ce phénomène est de plus en plus présent. En particulier sur les objectifs à ultra-grand angle d'entrée de gamme. Ce TIPE se penche donc sur la correction semi-automatique de ce problème avec la détection du coefficient de distorsion.

Le principe de l'algorithme repose sur une recherche itérative du paramètre de distorsion. Il s'ancre donc bien dans le thème "Cycles, Boucles".

Positionnement thématique

-INFORMATIQUE (Informatique pratique)

Mots-clés (en français)

- Distorsion d'image*
- Flou Gaussien*
- Filtre de Canny*
- Modèle de division pour la distorsion*
- Transformée de Hough*

Mots-clés (en anglais)

- Image distortion*
- Gaussian blur*
- Canny filter*
- Division model for distortion*
- Hough transform*

Problématique retenue

Comment déterminer la distorsion d'une image grâce aux lignes présentes ?

Objectifs du TIPE du candidat

Mon sujet de TIPE traitera les axes suivants :

- Détecter les contours d'une image
- Détecter les lignes sur une image avec distorsion
- Détecter le coefficient de distorsion d'une image
- Corriger l'image avec le coefficient détecté
- Cerner les limites de l'implémentation proposée

Bibliographie commentée

La distorsion d'image est un problème dans le domaine de la photographie, qui dégrade les images prises à travers une lentille par le capteur. Avec la démocratisation des smartphones, ce problème est présent pour un grand nombre de personnes au quotidien. Les aberrations optiques, dues aux lentilles ou à un mauvais alignement de la lentille avec le capteur, peuvent être la cause de ces distorsions. De plus, d'autres aberrations chromatiques et monochromatiques peuvent apparaître si des rayons ne se trouvent plus dans les conditions d'approximation de Gauss, notamment aux bords de la lentille. Ces défauts sont donc pratiquement inévitables. Dès lors, la question de leur correction se pose. Le modèle de division pour la distorsion est précis pour les distorsions radiales, permettant de décrire le changement de position des points sur l'image après distorsion [5, 6].

Corriger la distorsion peut se faire en détectant les lignes courbées qui étaient normalement droites, ce qui est caractéristique pour cette aberration, pour ensuite déterminer le paramètre de distorsion. Cette méthode a l'avantage d'être automatisable [2].

L'algorithme est composé de plusieurs opérations sur l'image.

Tout d'abord, un flou gaussien permet de réduire les bruits sur l'image ainsi que le nombre de contours détectés sur celle-ci [1,3].

Puis, un calcul du gradient indique les éléments de l'image où la variation locale de couleur est la plus forte. À partir de ces informations, il est possible de calculer la norme du gradient ainsi que sa direction [1,3].

Les contours sont ensuite construits de façon récursive avec un seuil haut, l'élément du gradient est alors considéré comme un élément d'un contour et un seuil bas, l'élément est considéré comme un contour que s'il est adjacent à un autre déjà classé comme tel [1].

Enfin, on applique la transformée de Hough sur les contours pour détecter les lignes. Cependant, la transformée à été modifiée pour prendre en compte la distorsion de chaque pixel. Ainsi, on itère plusieurs fois la recherche en faisant croître à chaque itération le paramètre de distorsion [1].

On obtient ainsi un paramètre avec un score maximal, qui sera considéré comme celui correspondant au coefficient de distorsion et utilisé pour appliquer la distorsion inverse sur l'image [4].

Cette méthode est donc efficace sur des images où des lignes droites sont clairement présentes. En revanche, pour les images qui en comportent peu ou si celles-là sont naturellement courbées, les résultats sont généralement insatisfaisants. C'est notamment le cas pour les images de paysage naturel. Par opposition, les photos urbaines contiennent en général beaucoup de lignes droites et ont donc des résultats plus fiables. Il est également possible d'avoir une base de donnée concernant les coefficients de distorsion de certains appareils photos, sans être bien évidemment exhaustive [2,4].

Références bibliographiques

- [1] DANIEL SANTANA-CEDRÉS, LUIS GOMEZ, MIGUEL ALEMÁN-FLORES, AGUSTIN SALGADO, JULIO ESCLARIN, LUIS MAZORRA, LUIS ALVAREZ *An Iterative Optimization Algorithm for Lens Distortion Correction Using Two-Parameter Models* <https://www.ipol.im/pub/art/2016/130/article.pdf> (Consultation le 13 Juin 2025)
- [2] MIGUEL ALEMÁN-FLORES, LUIS ALVAREZ, LUIS GOMEZ, DANIEL SANTANA-CEDRÉS *Automatic Lens Distortion Correction Using One-Parameter Division Models* <https://www.ipol.im/pub/art/2014/106/article.pdf> (Consultation le 13 Juin 2025)
- [3] WIKIPÉDIA *Canny edge detector* https://en.wikipedia.org/wiki/Canny_edge_detector (Consultation le 2 Septembre 2025)
- [4] WIKIPÉDIA *Distortion (optics)* [https://en.wikipedia.org/wiki/Distortion_\(optics\)](https://en.wikipedia.org/wiki/Distortion_(optics)) (Consultation le 30 Novembre 2025)
- [5] PIERRE DRAP, JULIEN LEFÈVRE *An Exact Formula for Calculating Inverse Radial Lens Distortions* <https://amu.hal.science/hal-01458821v1> (Consultation le 13 Juin 2025)
- [6] JOSÉ-PHILLIPPE PÉREZ *Optique, fondements et applications : avec 250 exercices et problèmes résolus : licence, classes préparatoires, Capes-agrégation* p. 97-102. ISBN : 2225852138

DOT

- [1] Avril 2025 : Recherche du sujet orienté sur le domaine du traitement numérique des images.
- [2] Mai - Juin 2025 : Recherches bibliographiques sur le sujet, découverte de la transformée de Hough, du modèle de distorsion de Brown-Conrady ainsi que celui de division.
- [3] Juillet - Août 2025 : Début de la programmation de l'algorithme, découverte la manipulation d'image en C. Implémentation du gradient et de la détection de contours pour une image. La vérification du bon fonctionnement de chaque étape était chronophage, le programme étant lent sur de grandes images, les résultats pouvaient être satisfaisants sur une image de petite dimension, mais non lorsqu'elle était plus conséquente.
- [4] Septembre - Octobre 2025 : Programmation de la transformée de Hough et premiers résultats avec des images modifiées simulant des cas réels. Plusieurs points techniques mal compris qui demandaient beaucoup de temps à cerner au vu de la quantité importante de variables. Avec la détection du paramètre de distorsion, il était alors possible d'effectuer la transformation inverse pour produire une image corrigée.
- [5] Novembre - Décembre 2025 : Débuts des tests avec plusieurs images modifiées et plusieurs paramètres de distorsion. Interprétation des résultats et modification de l'algorithme suivant les difficultés rencontrées en ajustant les données d'entrée du programme, pour que les résultats soient satisfaisants pour l'ensemble des images testées. Ajout de la fonction d'amélioration d'image pour la correction ainsi que des procédés automatiques qui facilitent l'exécution du programme.
- [6] Février 2026 : Validation des résultats précédents avec d'avantage d'images modifiées.